

Knapsack Problem
Dynamic Programming IV
(ADA-2023, CSE222)

Diptapriyo Majumdar

February 16, 2023

What we have seen so far

- ① Introduction and Recurrences.
- ② Divide and Conquer.
 - Counting Inversions in array.
 - Closest Pair of Points in 2D.
 - Selection of k 'th smallest in linear time (deterministic).
- ③ Dynamic Programming
 - Longest Common Subsequence.
 - Maximum Weight Independent Set in a Tree.
 - Longest Increasing Subsequence in an array.
 - Matrix Chain Multiplication

Knapsack Problem

Knapsack Problem:

- **Input:** There are n items $x_1, \dots, x_n$ each having positive values (or profits) $v_1, \dots, v_n$ respectively and positive weights $w_1, \dots, w_n$. In addition there is a number W called maximum weight (or Knapsack capacity).
- **Goal:** Compute a set of items $S \subseteq \{x_1, \dots, x_n\}$ such that $\sum_{x_i \in S} v_i$ (total profit) is maximized but $\sum_{x_i \in S} w_i \leq W$.
- **Input size:** There are n items each having values (profits) and weights, but W is something independent of all these and that has to be represented by $\log_2 W$ bits. Hence, the total input size is $\mathcal{O}(n \log_2 n) + \log_2 W$.

$$\mathcal{O}(n) + \log_2 w$$

Example

$$W = 11$$

Item	Value	Weight
x_1	1	1
x_2	6	2
x_3	18	5
x_4	22	6
x_5	28	7

Example

$W = 11$ $\rightarrow$ knapsack capacity.

Item	Value	Weight
x_1	1 v_1	1 w_1
x_2	6 v_2	2 w_2
x_3	18 v_3	5 w_3
x_4	22 v_4	6 w_4
x_5	28 v_5	7 w_5

- $\{x_1, x_2, x_5\}$ has total value 35 with knapsack capacity 10.
- $\{x_3, x_4\}$ has total value 40 with knapsack capacity 11.

Possible attempts

- Let $\text{OPT}(i)$ be the weight of an optimal solution for $\{x_1, \dots, x_i\}$. Consider two cases. $\text{OPT}(n)$
- **Case 1:** Let x_n is not part of an optimal solution S for $\{x_1, \dots, x_n\}$. Then, S must be an optimal solution for $\{x_1, \dots, x_{n-1}\}$.
- **Case 2:** Let x_n is part of an optimal solution S for $\{x_1, \dots, x_n\}$. Then, $S \setminus \{x_n\}$ be an optimal solution for $\{x_1, \dots, x_{n-1}\}$ with knapsack capacity $W - w_n$.

S is an optimal solution for $\{x_1, \dots, x_{n-1}\}$ with capacity W .

- Just keeping track of $\text{OPT}(i)$ is not enough. We need to keep track of the knapsack capacity as well.
- We need to keep track of $\text{OPT}(i, wt)$ for $0 \leq i \leq n$ and $0 \leq wt \leq W$.

Subproblem Definition: $\text{OPT}(i, wt)$ is the total maximum value of a set of items $S \subseteq \{x_1, \dots, x_i\}$ such

that $\sum_{x_i \in S} w_i \leq wt$.

$$0 \leq wt \leq W$$

Base Case: $\text{OPT}(0, wt) = 0$ for all $0 \leq wt \leq W$

$\text{OPT}(i, 0) = 0$ for all $1 \leq i \leq n$.

Algorithm

- **Base case:** For all $0 \leq wt \leq W$, we have $\text{OPT}(0, wt) = 0$. There are 0 items in the input. Then, the only optimal solution is $\emptyset$. Also, for all $1 \leq i \leq n$, we have that $\text{OPT}(i, 0) = 0$ as no knapsack capacity is allowed. So, only optimal solution is $\emptyset$.

$\text{OPT}(i, wt) = ?$ when $i, wt > 0$

$\{x_1, x_2, \dots, x_{i-1}, x_i\}$

$\{0, 1, \dots, wt-1, wt\}$.

$$\text{OPT}(i, wt) = \max \begin{cases} \text{OPT}(i-1, wt - w_i) + v_i \\ \text{OPT}(i-1, wt) \end{cases}$$

If $x_i \in S$ (optimal)

$S \setminus \{x_i\}$

↓
an optimal

for $\{x_1, \dots, x_{i-1}\}$

s.t. capacity
 $wt - w_i$



Algorithm

- **Base case:** For all $0 \leq wt \leq W$, we have $\text{OPT}(0, wt) = 0$. There are 0 items in the input. Then, the only optimal solution is $\emptyset$. Also, for all $1 \leq i \leq n$, we have that $\text{OPT}(i, 0) = 0$ as no knapsack capacity is allowed. So, only optimal solution is $\emptyset$.
- For all $1 \leq i \leq n$ and for all $1 \leq wt \leq W$, recursive definition is the following. $\text{OPT}(i, wt) = \max\{\text{OPT}(i-1, wt), \text{OPT}(i-1, wt - w_i) + v_i\}$
- Finally, $\text{OPT}(n, W) = \max\{\text{OPT}(n-1, W), \text{OPT}(n-1, W - w_n)\}$.

Recurrence:

$$\text{OPT}(i, wt) = \max \begin{cases} \text{OPT}(i-1, wt) \\ \text{OPT}(i-1, wt - w_i) + v_i \end{cases}$$

Solution to the actual problem: $\text{OPT}(n, W)$

KnapSack($n, W, \{x_1, \dots, x_n\}, \{v_1, \dots, v_n\}, \{w_1, \dots, w_n\}$)

for $0 \leq i \leq n$ do

 | Table($i, 0$) = 0;

end

for $0 \leq wt \leq W$ do

 | Table($0, wt$) = 0;

end

for $i = 1, \dots, n$ do

 for $wt = 1, \dots, W$ do

 if $w_i \leq wt$ then

 | Table[i, wt] = max{Table[$i - 1, wt$], $v_i +$ Table[$i - 1, wt - w_i$]};

 end

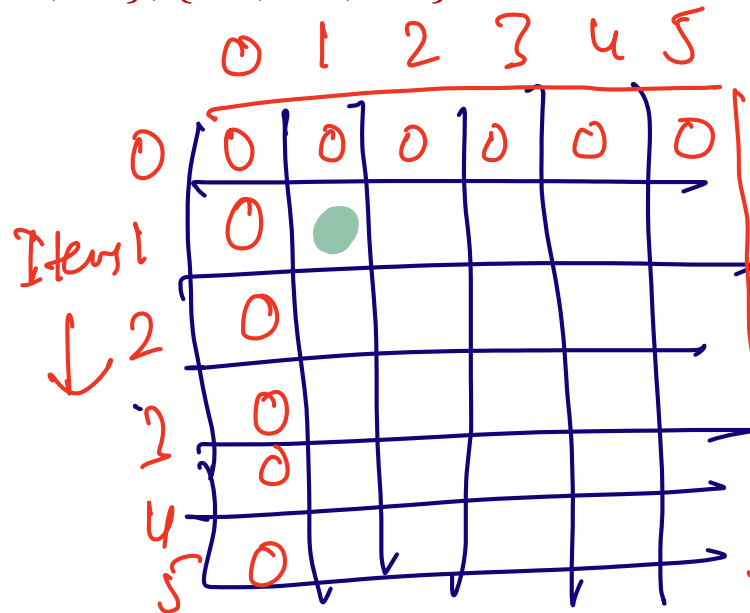
 else

 | Table[i, wt] = Table[$i - 1, wt$];

 end

end

end



$O(nW)$

Output Table[n, W];

Running time and Analysis

Running time: $O(nW)$

$\{x_1, \dots, x_n\}$ values $\{v_1, \dots, v_n\}$

W

weights $\{w_1, \dots, w_n\}$.

Input size

$$\hookrightarrow O(n) + \log_2 W$$

If W is $\text{poly}(n)$, i.e. $W = n^2$

then Input size is $O(n) + O(\log n) \approx O(n)$

Then this algorithm runs in polynomial-time.

But if $W \approx 2^n$, input size $O(n)$.

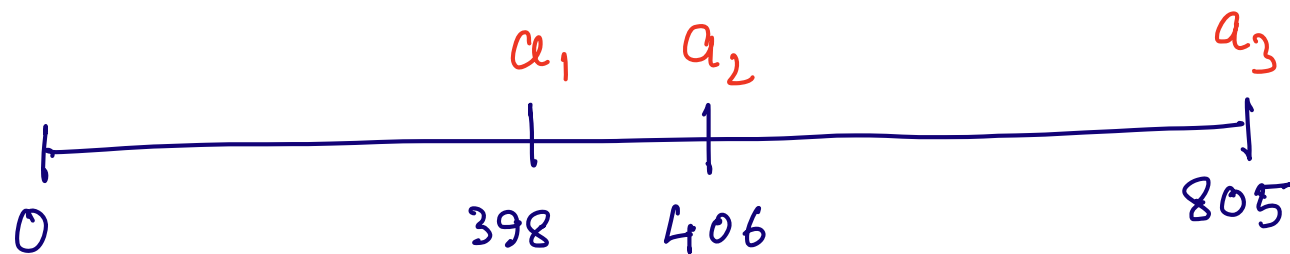
The algorithm runs in $O(n 2^n)$ time.

As W can be represented in binary bits, this algorithm for Knapsack is called PSEUDO-POLYNOMIAL TIME ALGORITHM.

A practice problem

- You are going on a long drive. You start at a kilometer post 0. Along the road that you will travel, there are n hotels at kilometer posts $a_1 < a_2 < \dots < a_n$. The values a_i s are measured from the starting point. When you choose to stop, you must stop at a hotel. Also you must stop at the last hotel that is your destination. You should ideally travel 400 Km per day. But you can only stop at hotels so that you may be forced to travel x Km on some specific date. But, there is a penalty $(400 - x)^2$ if you travel x Km on a specific day. Design a dynamic programming of your algorithm that outputs minimum total ~~cost~~ of the entire journey.

penalty



Subproblem definition:

$\text{cost}(i)$ = The minimum total penalty to reach the i -th hotel.

Recurrence:

$$\left\{ \begin{array}{l} \text{cost}(0) = 0 \\ \text{cost}(1) = (400 - a_1)^2 \end{array} \right\} \text{Base Cases.}$$

$$\text{cost}(i) = \min_{0 \leq j \leq i-1} \text{cost}(j) + \left[400 - (a_j - a_i) \right]^2$$

Solution for actual Problem: $\text{cost}(n)$

Algorithm outline:

Initialize $\text{cost}(0) = 0$
 $\text{cost}(1) = (400 - a_1)^2$.

For all $i = 2, \dots, n$

$$\text{cost}(i) = \min_{0 \leq j < i} \left\{ \text{cost}(j) + \left[400 - (a_j - a_i) \right]^2 \right\}$$

Output $\text{cost}(n)$.

Solution Sketch

- **Subproblem definition:** $Cost(i)$ is the minimum total cost when you start at 0 and end at i -th hotel.
- **Recurrences:**
- **Base Case:** $Cost(0) = 0$ and $Cost(1) = (400 - a_1)^2$.
- For all $k \geq 2$, $Cost(k) = \min_{0 \leq i \leq k-1} (Cost(i) + (400 - (a_k - a_i)^2))$.
- **Solution to the actual problem:** $Cost(n)$.
- **Algorithm outline:**
 - Initialize $Cost[0] = 0$ and $Cost[1] = (400 - a_1)^2$.
 - For all $k \geq 2$,
 $Cost[k] = \min_{0 \leq i \leq k-1} (Cost(i) + (400 - (a_k - a_i)^2))$.
 - Output $Cost[n]$.

Running time: $O(n^2)$.